



Machine Learning (MLOps) Investor Driven Market Analysis

Team No: 4

Name	USN
Manoj Malipatil	1RV23AI051
Maheshkumar	1RV23AI050
Mayur Kumar K N	1RV23AI054
Machani Bhanu Teja	1RV23AI049



Introduction

- Financial markets are driven by both price dynamics and investor sentiment from news and public discourse.
- The system integrates time-series market indicators with sentiment-based features for continuous analysis.
- An end-to-end MLOps pipeline ensures reliable data ingestion, preprocessing, feature extraction, training, evaluation, and tracking.
- Operational challenges such as streaming data, schema consistency, missing sentiment, and reproducibility are systematically addressed.
- The approach supports SDG 9 (Industry, Innovation, and Infrastructure) by enabling robust, transparent, and resilient financial analytics systems.

Motivation

The "Human" Factor

Financial markets are complex adaptive systems influenced not just by past prices, but by collective **investor sentiment**.

- Traditional models often ignore the qualitative "mood" of the market (Fear, Greed, Uncertainty).
- Operationalizing sentiment analysis requires robust pipelines to handle asynchronous, unstructured data.
- **Goal:** Capture the relationship between public discourse and market movement.





Problem Defination

Financial markets are influenced by both quantitative price movements and investor sentiment expressed through news and public information. However, traditional market monitoring approaches largely rely on historical price data and technical indicators, limiting their ability to capture sentiment-driven market behavior. While sentiment analysis techniques exist, their practical integration with market data is challenged by the unstructured nature of textual information, temporal misalignment, and inconsistent availability. Additionally, maintaining reliability in real-world deployment requires systematic handling of data consistency and model adaptability. Addressing these challenges is necessary for effective sentiment-aware market monitoring.



Objectives

Primary Objectives

- Analyze market behavior by incorporating sentiment signals with traditional indicators.
- Design an end-to-end MLOps workflow for continuous monitoring.
- Ensure reliable system operation through automated pipelines and versioning

Secondary Objectives

- Align sentiment features temporally with market time-series (lag analysis).
- Implement experiment tracking and model versioning.
- Monitor for data drift and trigger controlled retraining.



Dataset Description

Structured Data (Market)

Historical stock prices for **AAPL, GOOGL, TSLA**.

Features: Open, High, Low, Close, Volume, Daily Returns.

Unstructured Data (Sentiment)

Textual data from **Financial News & Reddit**.

Processing: Cleaning, Lowercasing, VADER Sentiment Analysis (Compound Scores).





Risk Analysis

Risk	Probability	Impact	Mitigation Strategy
Model Drift	Possible	Moderate	Periodic retraining on updated datasets; Drift detection (River).
Data Delay	Likely	High	Robust ingestion pipelines; Asynchronous processing.
Bad Prediction	Likely	High	Ensemble models (Random Forest); Larger diverse datasets.
Pipeline Failure	Possible	Major	Detailed logging (Prometheus); CI/CD validation.



Risk Assessment Matrix

	Very Low	Low	Moderate	High	Very High
Very Likely		Data Delay			
Likely		Biased Predictions		Bad Prediction	Model Unavailability
Possible			Model Drift		Pipeline Failure
Unlikely					
Rare			Version Incompatibility	Data Privacy Violation	Security Vulnerability Skill Loss



Tools Used

Tool	Role	Justification
DVC	Data & Pipeline Versioning	Enables reproducible, automated ML pipelines without bloating Git.
MLflow	Experiment Tracking	Records metrics and artifacts for systematic model comparison.
River	Online Drift Detection	Detects distribution shifts in streaming financial and sentiment data.
Docker	Containerization	Ensures environment consistency and portable deployment.
Prometheus	System Monitoring	Tracks operational metrics and inference activity.
Git	Source Control	Manages code evolution and integrates with DVC.



**RV College of
Engineering®**

Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Go, change the world

Phase - I

Machine Learning Models



Sentiment Processing

- ✓ **Scoring:** Used VADER analyzer to generate compound polarity scores $[-1, 1]$.
- ✓ **Aggregation:** Daily average of sentiment across all textual sources to match stock frequency.
- ✓ **Alignment:** Forward-filled missing values to maintain continuity. Created **Lagged Sentiment (t-1)** to model delayed market reaction.

Cumulative Returns Calculation

To evaluate long-term performance, we computed cumulative returns using the formula:

$$C_t = \prod_{i=1}^t (1 + r_i) - 1$$

Where r_i denotes the daily return at time i .

EDA

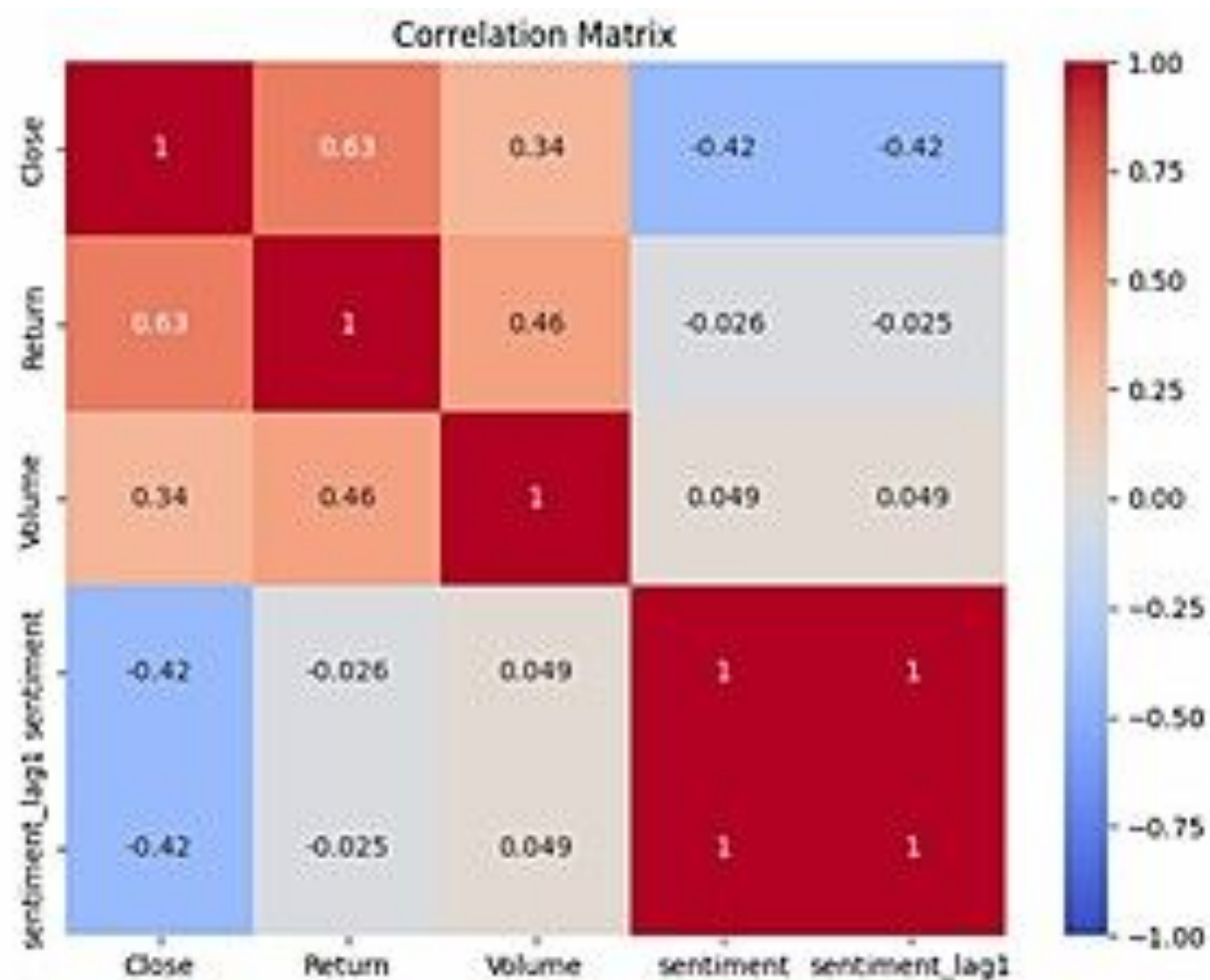
Key Insights

Sentiment Lag: Direct correlation between sentiment and same-day price is weak. However, *lagged* sentiment shows stronger correlation with returns, indicating a delayed market reaction.

Volatility Profiles:

- ✓ **TSLA:** High volatility, sharp growth phases.
- ✓ **AAPL/GOOGL:** More stable cumulative performance.

Design Decision: Due to distinct behavioral patterns, we train separate models per ticker rather than a single unified model.

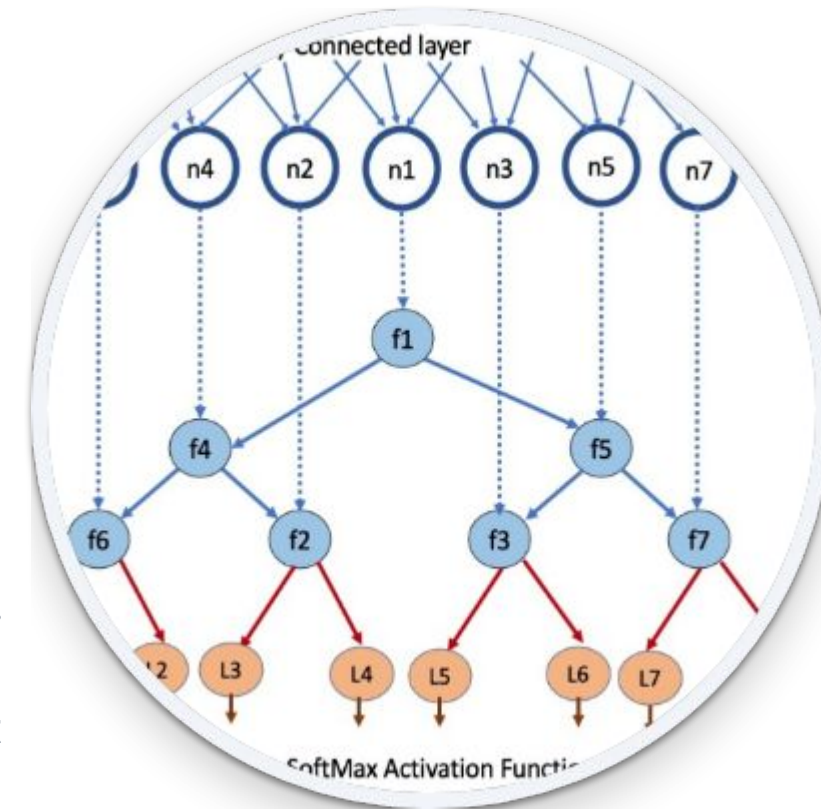


Algorithm Selection

Candidate Models

- ✓ **Random Forest (RF):** Ensemble method. Handles non-linear relationships and noisy data well. Less prone to overfitting on small datasets.
- ✓ **Multilayer Perceptron (MLP):** Deep learning approach. Sensitive to feature scaling and volatility.
- ✓ **LSTM:** Captured temporal dependencies but computationally heavier.

Selection: Models were trained independently for each stock ticker to account for individual volatility profiles.

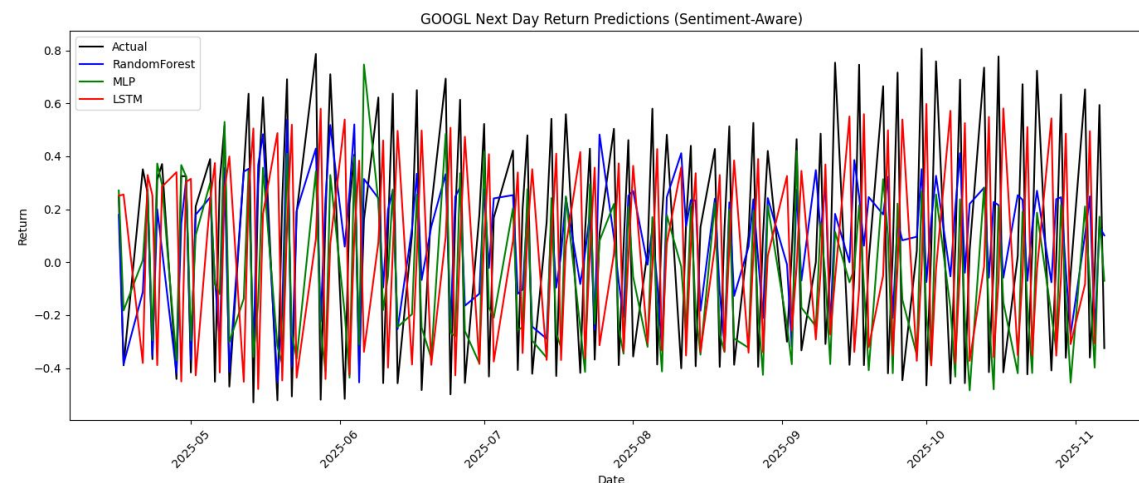


Model Performance Evaluation

Evaluation Strategy

Models were trained independently for each ticker using engineered features. The results highlighted key distinctions in model capability:

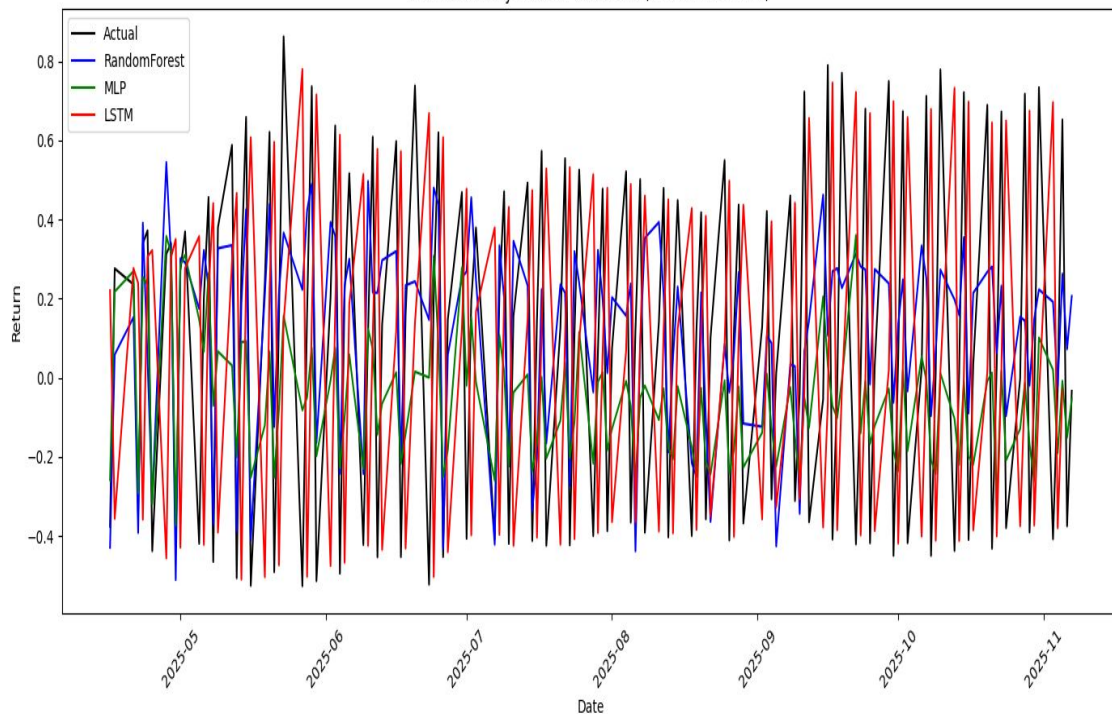
- ✓ **Stability:** Random Forest models consistently outperformed baseline neural models (MLP).
- Generalization:** Ensemble methods showed superior generalization on the heterogeneous dataset (Price + Sentiment).
- ✓ **Volatility Impact:** Performance varied significantly across stocks, with **TSLA** exhibiting higher prediction error due to its inherent market volatility.



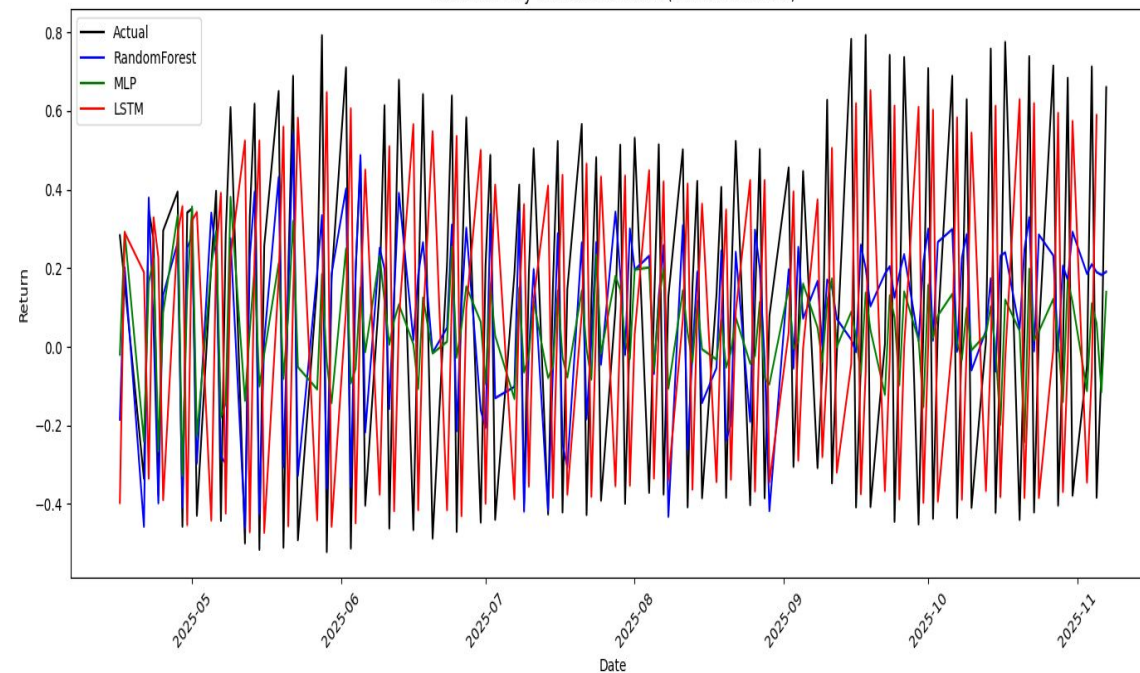


Model Performance Evaluation

AAPL Next Day Return Predictions (Sentiment-Aware)



TSLA Next Day Return Predictions (Sentiment-Aware)





Qualitative Analysis

Primary Metric: **Root Mean Squared Error (RMSE)** was chosen as it penalizes larger prediction errors, critical for risk-sensitive markets.

Ticker	RF RMSE (Random Forest)	MLP RMSE (Neural Network)	Performance Gap
AAPL	0.0633	0.4013	Significant (RF Superior)
GOOGL	0.0849	0.2510	Moderate
TSLA	0.1037	0.2460	High Volatility Impact

Note: Lower RMSE indicates better performance.



Why Random Forest Won?

Robustness vs. Sensitivity

The quantitative results clearly indicate that Random Forest models outperform Multilayer Perceptrons across all three assets.

-  **Robustness:** Ensemble-based methods (RF) are highly effective at handling noisy and heterogeneous features, such as combining sentiment scores with financial indicators.
-  **Sensitivity:** MLP models proved more sensitive to feature scaling and data volatility, leading to higher errors, especially in volatile stocks like TSLA.



Phase II – MLOPs Life Cycle

	Component	MLOps Tools
1	Data Management	DVC, Git (for metadata tracking)
2	Experiment Tracking and Version Control	MLFlow, DVC
3	Drift Detection	River: Adwin Drift, Page Hinkley Drift
3	Automation and Pipeline Orchestration	DVC Pipeline, GitHub Codespaces
4	Model Deployment & Serving	Docker, Kubernetes, Hugging Face Hub Spaces
5	Monitoring & Governance and collaboration	Prometheus, Github

Data Management

Unified Data Engineering

Effective MLOps begins with a robust ingestion strategy for heterogeneous data sources.

Sentiment Distribution



- ✓ **Ingestion:** Multi-source pipeline handling structured market data (OHLCV) and unstructured textual streams (News, Social Media).
- ✓ **Preprocessing:** Text cleaning (URL removal, lowercasing) and sentiment extraction using the VADER analyzer.
- ✓ **Feature Engineering:** Creation of temporal features, specifically **lagged sentiment (t-1)**, to align public discourse with market reaction.
- ✓ **Versioning:** Utilizing **DVC** to track dataset versions, ensuring every model can be traced back to its specific training data.

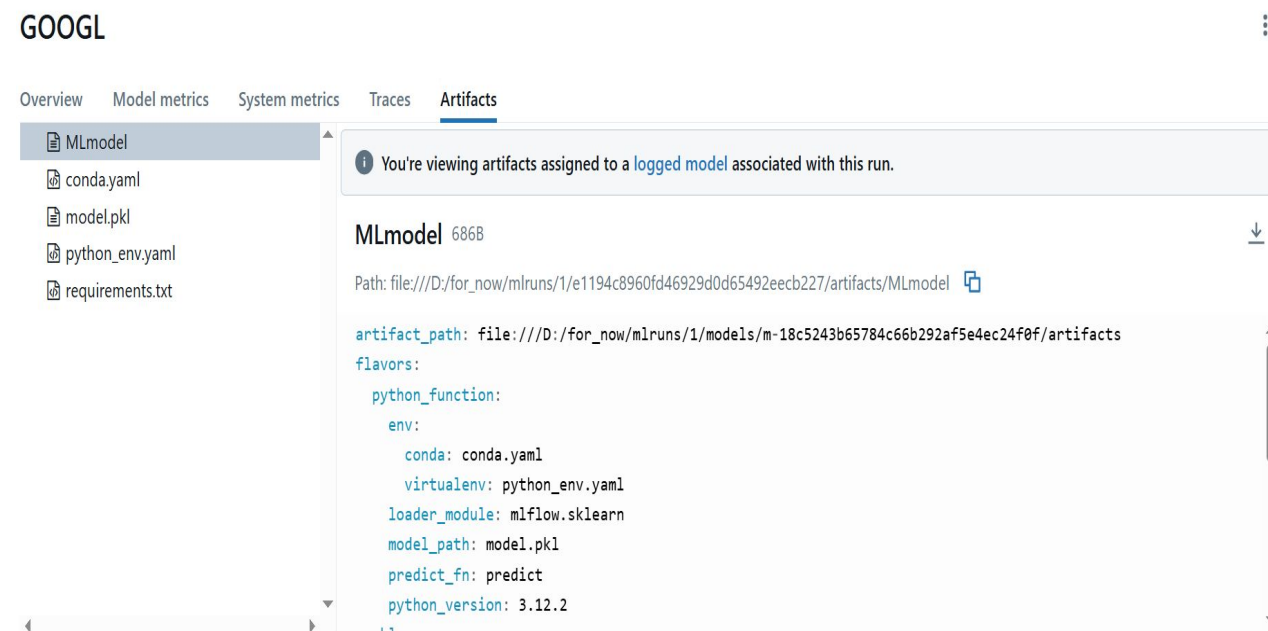


Experiment Tracking & Version Control

Systematic Reproducibility

We moved beyond ad-hoc modeling by implementing rigorous tracking standards.

- ✓ **MLflow:** Serves as the central repository for logging hyperparameters, evaluation metrics (RMSE, MAE), and trained model artifacts.
- ✓ **Comparison:** Enables apples-to-apples comparison between baseline Neural Networks (MLP) and Random Forest models.
- ✓ **Full Lineage:** The combination of **Git** (code) and **DVC** (data/models) ensures that any historical experiment can be perfectly reproduced.





Experiment Tracking & Version Control

☐	Run Name	Created ↕	Dataset	Duration	Source	Models
☐	TSLA	✓ 10 days ago	-	17.9s	train_mo...	mlp +1
☐	GOOGL	✓ 10 days ago	-	18.1s	train_mo...	mlp +1
☐	AAPL	✓ 10 days ago	-	26.9s	train_mo...	mlp +1
☐	AAPL	✓ 10 days ago	-	26.8s	train_mo...	mlp +1
☐	AAPL	✓ 10 days ago	-	29.9s	train_mo...	mlp +1



Automation & Orchestration

The DVC Pipeline

The system is orchestrated as a Directed Acyclic Graph (DAG) defined in `dvc.yaml`, enabling intelligent dependency management.

- ✓ **Workflow Stages:** Ingestion → Feature Engineering → Training → Drift Detection → Retraining.
- ✓ **Conditional Logic:** A closed-loop feedback mechanism where model retraining is triggered *automatically* only when statistically significant drift is detected.
- ✓ **Efficiency:** The pipeline skips execution for stages where inputs haven't changed, optimizing computational resources.

```
$ python -m dvc repro
```

```
@Mayur-cinderace → /workspaces/Investor-Sentiment-Aware-Market-Monitoring
Stage 'ingest' didn't change, skipping
Stage 'features' didn't change, skipping
Stage 'train' didn't change, skipping
Stage 'evaluate' didn't change, skipping
Stage 'drift' didn't change, skipping
Stage 'retrain_decision' didn't change, skipping
Stage 'retrain' didn't change, skipping
Data and pipelines are up to date.
```


Model Deployment & Serving

Production-Ready Inference

Deployment focuses on reliability, portability, and low-latency serving.

Investor Sentiment Inference API 0.1.0 OAS 3.1

/openapi.json

default

POST

/predict

Predict

GET

/metrics

Metrics

GET

/health

Health

Schemas

HTTPValidationError > Expand all object

InputText > Expand all object

ValidationError > Expand all object

- ✓ **FastAPI:** A high-performance RESTful API handles real-time requests, converting raw text into sentiment scores and generating market predictions on the fly.
- ✓ **Docker:** Containerization encapsulates the model, dependencies, and runtime, ensuring "write once, run anywhere" consistency between dev and prod.
- ✓ **Cloud-Native:** The stateless architecture supports seamless scaling on cloud platforms like GitHub Codespaces or Kubernetes.



Model Deployment & Serving

```
@Mayur-cinderace → /workspaces/Investor-Sentiment-Aware-Market-Monitoring (main) $ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
investor-api-76898c5cf8-lrs2j	1/1	Running	2 (130m ago)	6h23m

 mayur1313
Docker Personal

Repositories

Hardened Images

Collaborations

Settings

Default privacy

Notifications

Billing

Usage

Pulls

Storage

Repositories

All repositories within the mayur1313 namespace.

All content Create a repository

Name	Last Pushed	Contains	Visibility	Scout
mayur1313/investor-mlops-api	about 7 hours ago	IMAGE	Public	Inactive

1-1 of 1



Monitoring, Governance & Collaboration

Holistic Observability

We maintain system trust through comprehensive monitoring of both data health and infrastructure performance.

- ✓ **Prometheus:** Scrapes system-level metrics (latency, request counts, error rates) to ensure API health.
- ✓ **River:** Provides real-time online concept drift detection on streaming sentiment data.
- ✓ **Collaboration:** A **Streamlit** UI allows stakeholders to interact with the model, while Git ensures code governance and auditability.

```
{  
  "return_lag1": {  
    "adwin_drift": false,  
    "page_hinkley_drift": false,  
    "drift_flag": false  
  },  
  "volume_lag1": {  
    "adwin_drift": false,  
    "page_hinkley_drift": false,  
    "drift_flag": false  
  },  
  "sentiment_lag1": {  
    "adwin_drift": false,  
    "page_hinkley_drift": false,  
    "drift_flag": false  
  }  
}
```



Monitoring, Governance & Collaboration

Simulated real-time stream with ML inference and monitoring

☒ Auto-refresh (live stream)

Refresh interval (seconds)

5

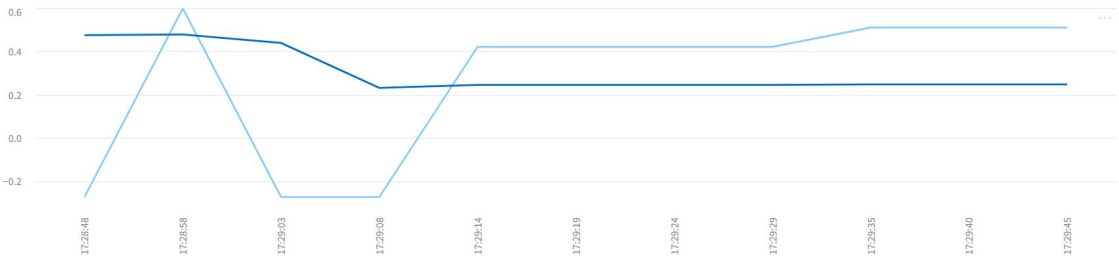
Live Predictions

Latest Sentiment

0.511

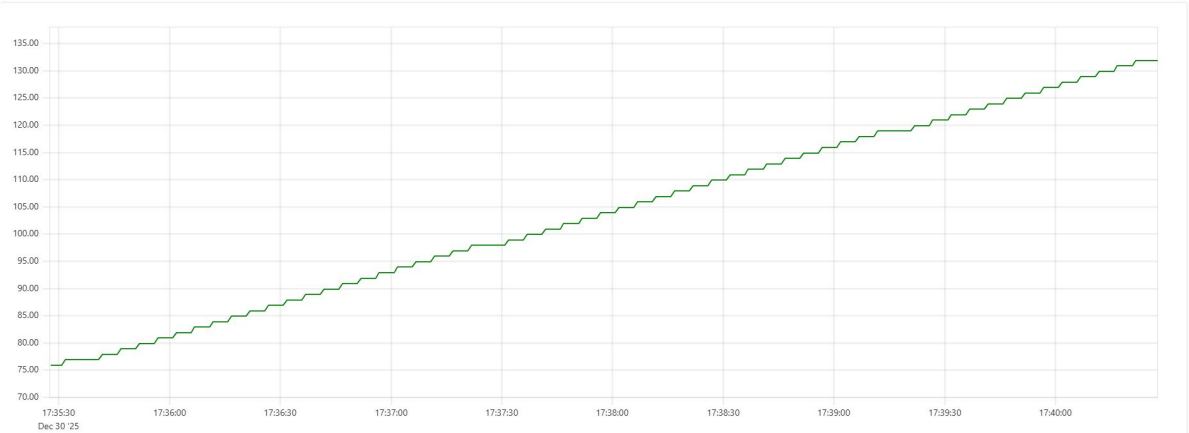
Predicted Return

0.2475

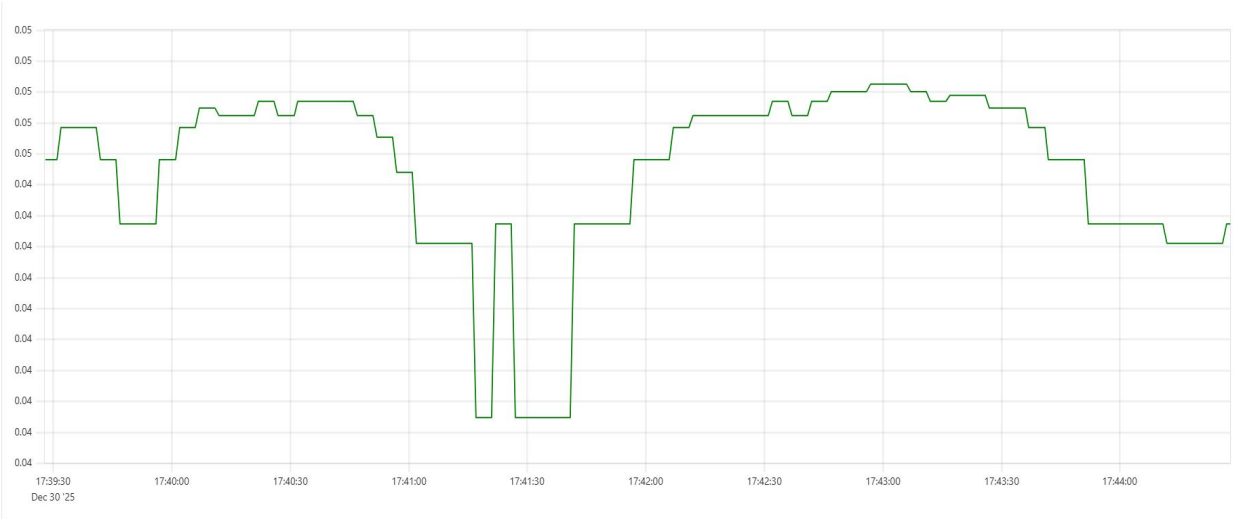


Sentiment and Predicted return plots

Total prediction requests vs time

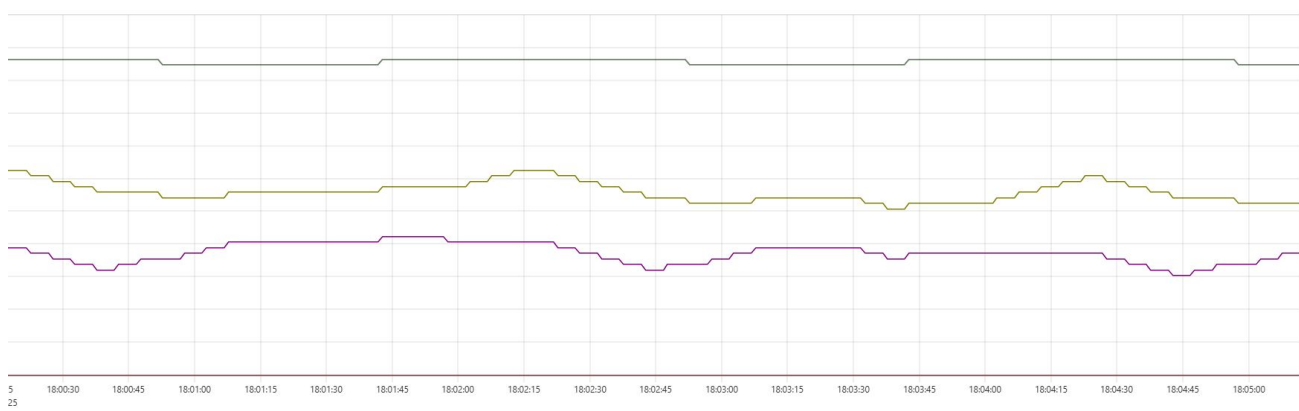


Monitoring, Governance & Collaboration



Model latency modelled using a histogram

Sentiment distribution bucket frequency





**RV College of
Engineering®**

Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Go, change the world

Thank You